DATA STRUCTURES PROJECT REPORT

EXPRESSION EVALUATOR WITH SCIENTIFIC CALCULATOR

Project Report

Project Members

Name	Roll Number
Anesh L	2025115115
Ganesh Kumar	2025115077
Monesh Udhyai S	2025115113

Programming Language: C

Project Area: Data Structures — Stacks and Arrays

ABSTRACT

The Expression Evaluator with Scientific Calculator is a Data Structures project developed using the C programming language. The main objective is to evaluate mathematical expressions containing arithmetic operators, variables, and scientific functions. The project uses stacks and arrays to perform expression conversion and evaluation.

The program converts an infix mathematical expression into postfix notation using a stack. During conversion, operator precedence and parentheses are handled to ensure the correct order of operations. The postfix expression is then evaluated using a separate stack that stores numerical values. The program supports addition, subtraction, multiplication, division, modulus, and exponentiation. It also supports sine, cosine, tangent, square root, and logarithm. Trigonometric functions accept angles in degrees, and users can enter values for variables during evaluation.

Arrays store the input expression, operator stacks, postfix expression, and numerical results. Stack operations such as push and pop are used during conversion and evaluation. The project demonstrates stack implementation using arrays, infix-to-postfix conversion, operator precedence, and postfix expression evaluation.

TABLE OF CONTENTS

1	Introduction
2	Problem Statement and Objectives
3	Scope and Requirements
4	Data Structures and Concepts
5	System Design
6	Algorithms
7	Implementation
8	Testing and Results
9	Conclusion

1. INTRODUCTION

Mathematical expressions are widely used in computing and scientific applications. Expressions may contain several arithmetic operators, parentheses, variables, and mathematical functions. To evaluate them correctly, a program must follow operator precedence and the order indicated by parentheses.

A stack is a linear data structure that follows the Last In, First Out (LIFO) principle. The last element inserted is the first element removed. Stacks are useful in expression processing because they can temporarily store operators during conversion and numerical values during evaluation.

This project implements an expression evaluator in C using stacks and arrays. The expression is first converted from infix notation to postfix notation and then evaluated using a numerical stack.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Expressions with multiple operators and parentheses can be evaluated incorrectly if the order of operations is not handled properly. The problem is to develop a program that can process such expressions, support variables and scientific functions, and calculate the result using suitable data structures.

2.2 Objectives

- Develop an expression evaluator using the C programming language.
- Convert infix expressions into postfix notation using a stack.
- Handle operator precedence and parentheses correctly.
- Evaluate postfix expressions using a numerical stack.
- Support addition, subtraction, multiplication, division, modulus, and exponentiation.
- Support sine, cosine, tangent, square root, and logarithm functions.
- Allow users to enter values for variables.
- Demonstrate the practical use of stacks and arrays in mathematical computation.

3. SCOPE AND REQUIREMENTS

3.1 Scope

The project evaluates mathematical expressions containing arithmetic operators, variables, and selected scientific functions. Trigonometric functions accept angles in degrees. Arrays are used to store expressions and intermediate values, while stacks support conversion and evaluation.

3.2 Functional Requirements

- Accept a mathematical expression from the user.
- Convert the infix expression to postfix notation.
- Apply operator precedence and parentheses rules.
- Accept values for variables used in an expression.
- Evaluate the postfix expression using a numerical stack.
- Perform the supported arithmetic and scientific operations.
- Display the final calculated result.

3.3 Non-Functional Requirements

- Correctness: the expression must be evaluated in the intended order.
- Usability: prompts and results should be clear.

- Maintainability: conversion and evaluation should be organized into separate steps.
- Reliability: invalid expressions and mathematical errors should be handled appropriately.

4. DATA STRUCTURES AND CONCEPTS

4.1 Arrays

An array stores elements of the same data type in indexed memory locations. In this project, arrays store the input expression, operator information, postfix expression, and numerical results. Arrays also provide the storage needed for implementing stacks.

4.2 Stack Operations

- **Push:** inserts an element at the top of the stack.
- **Pop:** removes and returns the top element.
- **Peek:** reads the top element without removing it.
- **Empty check:** checks whether the stack has any elements.

An array-based stack uses an index, often called TOP, to identify the top element. Push and pop operations must check the stack capacity and ensure that the stack is not empty when removing an item.

4.3 Infix and Postfix Notation

In infix notation, the operator appears between operands, as in A + B. In postfix notation, the operator follows its operands, as in A B +. Postfix notation represents the evaluation order directly and simplifies the evaluation process.

4.4 Operator Precedence

Operator precedence determines the order in which operators are evaluated. Parentheses allow the user to specify the intended grouping. During conversion, operators are stored temporarily on a stack and moved to the postfix output according to precedence and associativity.

4.5 Scientific Functions

The calculator supports sine, cosine, tangent, square root, and logarithm. Trigonometric values are entered in degrees. The program should also consider mathematical domain restrictions, such as negative values supplied to a real square-root function or non-positive values supplied to a logarithm.

5. SYSTEM DESIGN

5.1 Working Principle

Stage	Description
Input	Read the expression and required variable values.
Conversion	Convert infix notation to postfix notation using an operator stack.
Evaluation	Evaluate postfix tokens using a numerical stack.
Output	Display the calculated result.

5.2 Main Modules

- **Input module:** accepts the expression and variable values.
- **Conversion module:** converts infix expressions into postfix notation.
- **Stack module:** performs push, pop, and related operations.

- **Evaluation module:** calculates the result from the postfix expression.
- **Scientific function module:** calculates the supported mathematical functions.
- **Output module:** displays the final result.

6. ALGORITHMS

6.1 Infix-to-Postfix Conversion

- Initialize an empty operator stack and postfix output.
- Scan the expression from left to right.
- Append operands and variable tokens to the postfix output.
- Push opening parentheses onto the stack.
- When a closing parenthesis is found, pop operators to the output until the matching opening parenthesis is reached.
- For an operator, pop operators of higher precedence and operators of equal precedence when required by associativity; then push the current operator.
- After scanning the expression, append the remaining operators to the output if the expression is valid.

6.2 Postfix Evaluation

- Initialize an empty numerical stack.
- Scan the postfix expression from left to right.
- Push each numeric operand onto the stack.
- Resolve variable tokens using the values entered by the user and push those values.
- When a binary operator is encountered, pop the required operands, perform the operation, and push the result.
- When a scientific function is encountered, apply it to the required argument and push the result.
- For a valid expression, the stack should contain one final result after evaluation.

6.3 Example

Consider the expression (2 + 3) * 4. Its postfix form is 2 3 + 4 *. The values 2 and 3 are combined by addition to produce 5. The value 4 is then multiplied by 5, producing the final result 20.

7. IMPLEMENTATION

The program is written in C. Arrays are used to store the expression and intermediate information. A stack is used during infix-to-postfix conversion to manage operators and parentheses. A separate numerical stack is used to evaluate the postfix expression.

7.1 Arithmetic Operations

The supported arithmetic operations are addition, subtraction, multiplication, division, modulus, and exponentiation. Division should check for a zero divisor. The implementation should also define how modulus is handled for the accepted operand types. In C, the operator ^ represents bitwise XOR rather than mathematical exponentiation, so exponentiation requires a suitable implementation.

7.2 Scientific Operations

Sine, cosine, and tangent use degree-based input. If the standard C math library is used, the degree value must be converted to radians before calling the trigonometric functions. Square root and logarithm operations should be checked for invalid input values.

7.3 Variables

The evaluator allows users to supply values for variables used in an expression. During evaluation, each variable must be associated with the correct numerical value before it can be used in an operation.

8. TESTING AND RESULTS

No.	Test expression	Expected result / check
1	2 + 3 * 4	14
2	(2 + 3) * 4	20
3	10 - 4 - 2	4, if subtraction is left-associative
4	sin(30)	0.5 when 30 is interpreted as degrees
5	sqrt(25)	5
6	log(1)	0 for a standard logarithm
7	Expression with a variable	Uses the value supplied by the user
8	Division by zero	Should be handled safely
9	Mismatched parentheses	Should be reported as invalid if validation is implemented

9. CONCLUSION

The Expression Evaluator with Scientific Calculator demonstrates the use of stacks and arrays in solving a practical mathematical problem. Infix-to-postfix conversion handles operator precedence and parentheses, while postfix evaluation uses a stack to calculate the result. The support for arithmetic operations, variables, and scientific functions makes the project a useful application of Data Structures using C.

Testing with different expressions helps verify the conversion and evaluation stages. The project illustrates how appropriate data structures can simplify the processing of mathematical expressions.